CSE 270 Sports Analytics

PSS 2: Predictions with Ratings

Narek Sahakyan

29-09-2022

Poisson Regression

Poisson Regression

```
nhl_games <- read.csv("../data/game.csv")
nhl_teams <- read.csv("../data/team_info.csv") %>%
  select(team_id, abbreviation)

nhl_games <- nhl_games %>%
  drop_na()

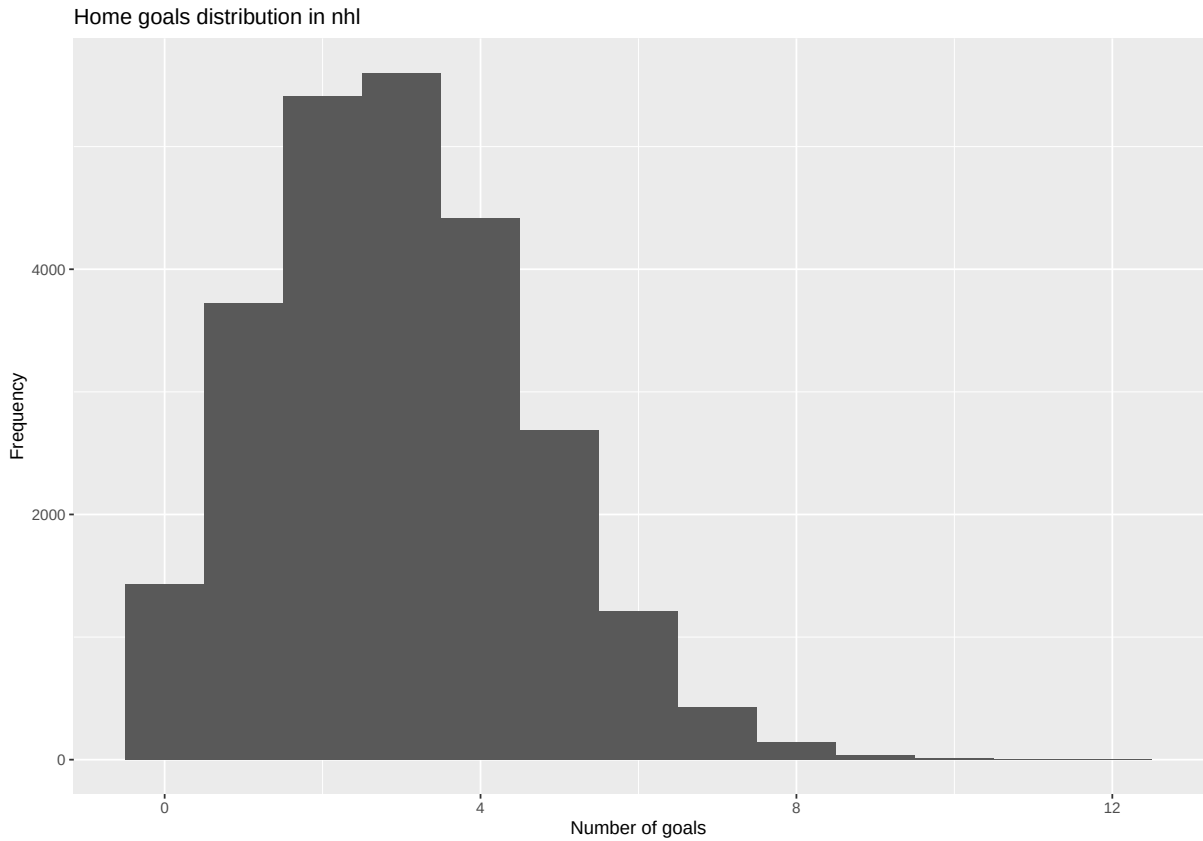
nhl_games <- nhl_games %>%
  inner_join(nhl_teams, by = c("home_team_id" = "team_id")) %>%
  rename("home_team_abr" = abbreviation)

nhl_games <- nhl_games %>%
  inner_join(nhl_teams, by = c("away_team_id" = "team_id")) %>%
  rename("away_team_abr" = abbreviation)
```

Poisson Regression

Checking that the home and away goals follow a poisson distribution in nhl. Home goals histogram

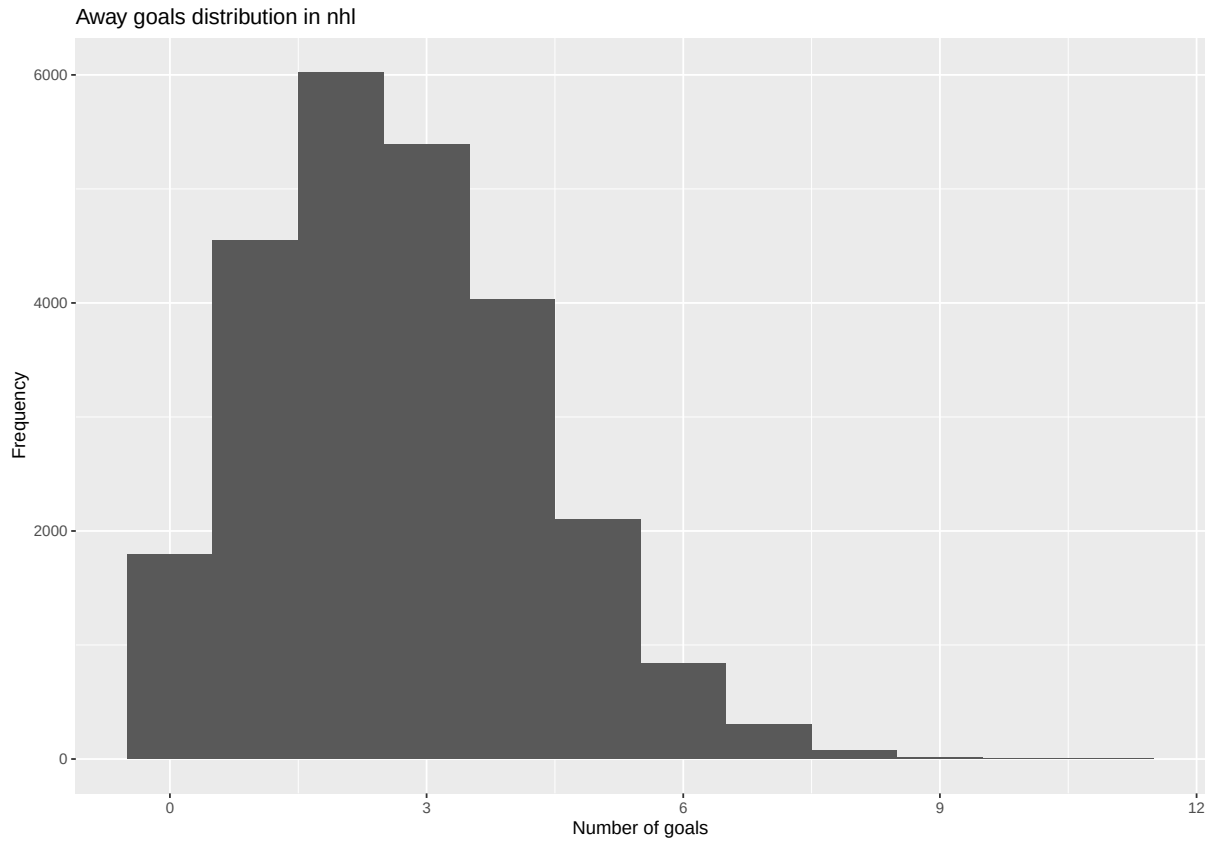
```
ggplot(nhl_games, aes(x = home_goals)) +
  geom_histogram(binwidth = 1) +
  ggtitle("Home goals distribution in nhl") +
  xlab("Number of goals") +
  ylab("Frequency")
```



Poisson Regression

Away goals histogram

```
ggplot(nhl_games, aes(x = away_goals)) +
  geom_histogram(binwidth = 1) +
  ggtitle("Away goals distribution in nhl") +
  xlab("Number of goals") +
  ylab("Frequency")
```



Poisson Regression

Dividing into test and train data sets.

```
nhl_games_train <- nhl_games %>%
  filter(season != "20192020")

nhl_games_test <- nhl_games %>%
  filter(season == "20192020")
```

Poisson Regression

Data modification.

```
get_poisson_form <- function(train_nhl_bta, country){

  nhl_home <- train_nhl_bta %>%
    group_by(home_team_abr) %>%
    mutate(TEAM = as.factor(home_team_abr),
           OPPONENT = as.factor(away_team_abr),
           GOAL = home_goals,
           HOME = 1) %>%
    ungroup() %>%
    select(TEAM, OPPONENT, GOAL, HOME)

  nhl_away <- train_nhl_bta %>%
    group_by(away_team_abr) %>%
    mutate(TEAM = as.factor(away_team_abr),
           OPPONENT = as.factor(home_team_abr),
           GOAL = away_goals,
           HOME = 0) %>%
    ungroup() %>%
    select(TEAM, OPPONENT, GOAL, HOME)

  return(rbind(nhl_home, nhl_away))
}

nhl_games_train_poisson <- get_poisson_form(nhl_games_train)
nhl_games_test_poisson <- get_poisson_form(nhl_games_test)
```

Poisson Regression

Fitting a model.

```
nhl_model_poisson <- glm(GOAL~TEAM+OPPONENT+HOME, data=nhl_games_train_poisson, fami
summary(nhl_model_poisson)
```

```
##
## Call:
## glm(formula = GOAL ~ TEAM + OPPONENT + HOME, family = poisson(link = log),
##     data = nhl_games_train_poisson)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.6992  -0.6910  -0.0101   0.6278   3.7222
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.0210002  0.0302834  33.715  < 2e-16 ***
## TEAMARI     -0.1039000  0.0329565  -3.153 0.001618 **
## TEAMBOS      0.0595836  0.0214541   2.777 0.005482 **
## TEAMBUF     -0.0120353  0.0222771  -0.540 0.589023
## TEAMCAR     -0.0208929  0.0222677  -0.938 0.348109
## TEAMCBJ     -0.0150186  0.0220895  -0.680 0.496568
## TEAMCGY      0.0250273  0.0219101   1.142 0.253339
## TEAMCHI      0.0695712  0.0214876   3.238 0.001205 **
```

Poisson Regression

Calculating RMSE as a prediction accuracy metric.

```
nhl_games_test_poisson$P.G <- predict(nhl_model_poisson,
                                      nhl_games_test_poisson, type = "response")

predict(nhl_model_poisson,data.frame(TEAM = "BOS", OPPONENT = "ATL",
                                     HOME = 1, type = "response"))
```

```
##        1
## 1.270123
```

```
sqrt(mean((nhl_games_test_poisson$GOAL - nhl_games_test_poisson$P.G)^2))
```

```
## [1] 1.690189
```

Poisson Regression

Underestimated and overestimated teams.

```
underestimated_nhl_poisson <- nhl_games_test_poisson %>%
  filter(GOAL < P.G)

overestimated_nhl_poisson <-nhl_games_test_poisson %>%
  filter(GOAL > P.G)
```

Bradley-Terry Model

Bradley-Terry Model

Data modification.

```
nhl_games_bt <- nhl_games %>%
  filter(home_goals != away_goals) %>%
  mutate(home_wl = ifelse(home_goals > away_goals, 'W', 'L'))

team_levels <- unique(c(nhl_games_bt$home_team_abr, nhl_games_bt$away_team_abr))

nhl_games_bt <- nhl_games_bt %>%
          select(season, home_team_abr, away_team_abr, home_wl) %>%
          mutate(ht_w = ifelse(home_wl=="W",1,0),
                 at_w = ifelse(home_wl=="L",1,0),
                 home_team_abr = factor(home_team_abr, levels = team_levels),
                 away_team_abr = factor(away_team_abr, levels = team_levels))
```

Bradley-Terry Model

Test and train datasets.

```
train_nhl_bt <- nhl_games_bt  %>%
        filter(season != "20192020") %>%
        group_by(home_team_abr, away_team_abr) %>%
        summarise(ht=sum(ht_w), at=sum(at_w))

test_nhl_bt <- nhl_games_bt  %>%
        filter(season == "20192020") %>%
        group_by(home_team_abr, away_team_abr) %>%
        summarise(ht=sum(ht_w), at=sum(at_w)) %>%
        filter(home_team_abr %in% (train_nhl_bt$home_team_abr),
        away_team_abr %in% (train_nhl_bt$away_team_abr))
```

Bradley-Terry Model

Fitting a model

```
BT_Model <- BTm(cbind(ht,at), home_team_abr, away_team_abr,
                data=train_nhl_bt, id="team_")
summary(BT_Model)
```

```
##
## Call:
## BTm(outcome = cbind(ht, at), player1 = home_team_abr, player2 = away_team_abr,
##     id = "team_", data = train_nhl_bt)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -3.2495  -0.2419   0.4624   1.1539   6.1360
##
## Coefficients:
##            Estimate Std. Error z value Pr(>|z|)
## team_BUF -0.2060658  0.0738709  -2.790 0.005278 **
## team_WPG  0.0207492  0.0891688   0.233 0.815997
## team_CAR -0.1767802  0.0738996  -2.392 0.016749 *
## team_ANA  0.0422070  0.0711865   0.593 0.553243
## team_MTL -0.0504383  0.0733281  -0.688 0.491551
## team_LAK -0.0410628  0.0711482  -0.577 0.563842
## team_COL  0.0006243  0.0713101   0.009 0.993015
```

Bradley-Terry Model

Predicting on test dataset.

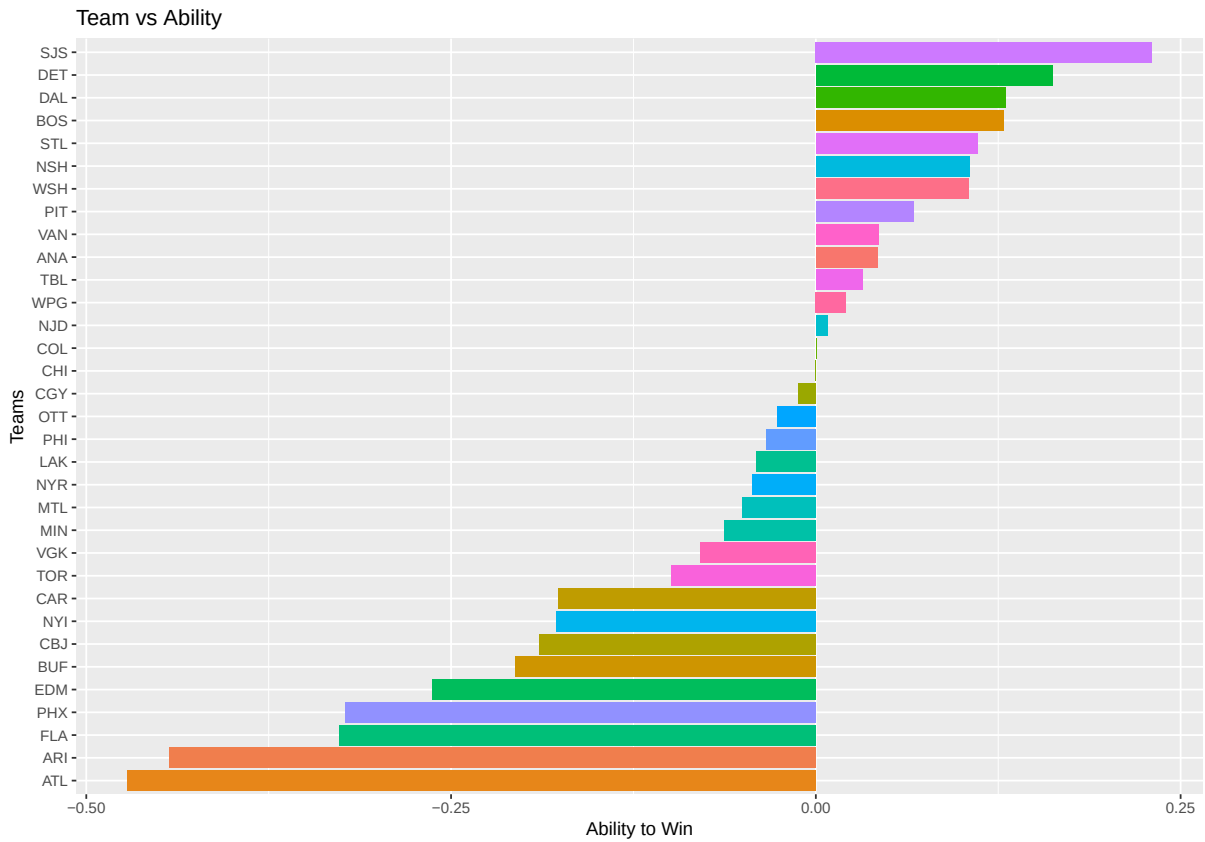
```
test_nhl_bt$home_win <- predict(BT_Model, test_nhl_bt)
test_nhl_bt$away_win <- 1 - test_nhl_bt$home_win
```

Bradley-Terry Model

Plotting the abilities.

```
abilities <- as.data.frame(BTabilities(BT_Model))
abilities$team <- rownames(abilities)
abilities <- abilities[order(abilities$ability, decreasing = T),]

ggplot(data=abilities, aes(x=reorder(team,ability), y = ability, fill =team))+ geom_bar(stat="identity") + coord_flip() + theme(legend
  labs(x="Teams", y="Ability to Win", title="Team vs Ability")
```



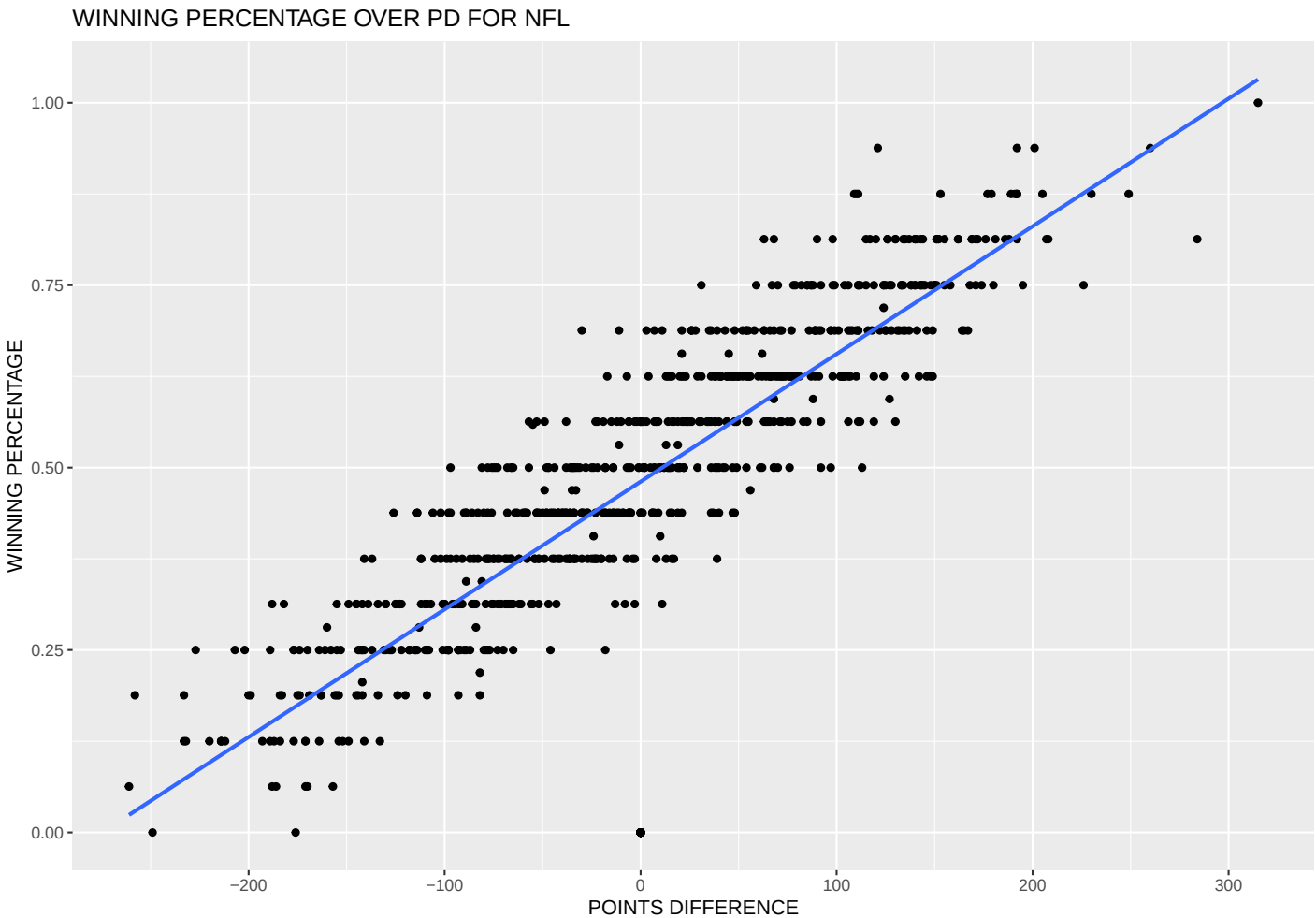
Pythagorean Wins

Pythagorean Wins

```
data("nfl_st")
nfl_st <- nfl_st %>%
  group_by(Season) %>%
  mutate(Rank = 1:n())
```

Pythagorean Wins

```
nfl_st %>%
  ggplot(aes(x = PD, y = Wpct)) +
  geom_point() +
  geom_smooth(method = "lm", se = F) +
  labs(x = "POINTS DIFFERENCE", y = "WINNING PERCENTAGE") +
  ggtitle("WINNING PERCENTAGE OVER PD FOR NFL")
```



Pythagorean Wins

Correlation between the variables.

```
cor.test(nfl_st$PD, nfl_st$Wpct)
```

```
##
##  Pearson's product-moment correlation
##
## data:  nfl_st$PD and nfl_st$Wpct
## t = 38.91, df = 761, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  0.7905750 0.8382189
## sample estimates:
##       cor
## 0.8157763
```

Pythagorean Wins

Fitting a model.

```
nfl_st_reg <- nfl_st %>%
  filter(W!= 0, L != 0, PA != 0) %>%
  mutate(W = W + 0.5 * T, L = L + 0.5 * T, RATIO = PF / PA)

model <- lm(log(W/L)~0+log(RATIO), data=nfl_st_reg)
summary(model)
```

```
##
## Call:
## lm(formula = log(W/L) ~ 0 + log(RATIO), data = nfl_st_reg)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.41011 -0.22623  0.01348  0.27157  1.63273
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## log(RATIO)  2.73311    0.04963   55.07   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3965 on 729 degrees of freedom
```

Pythagorean Wins

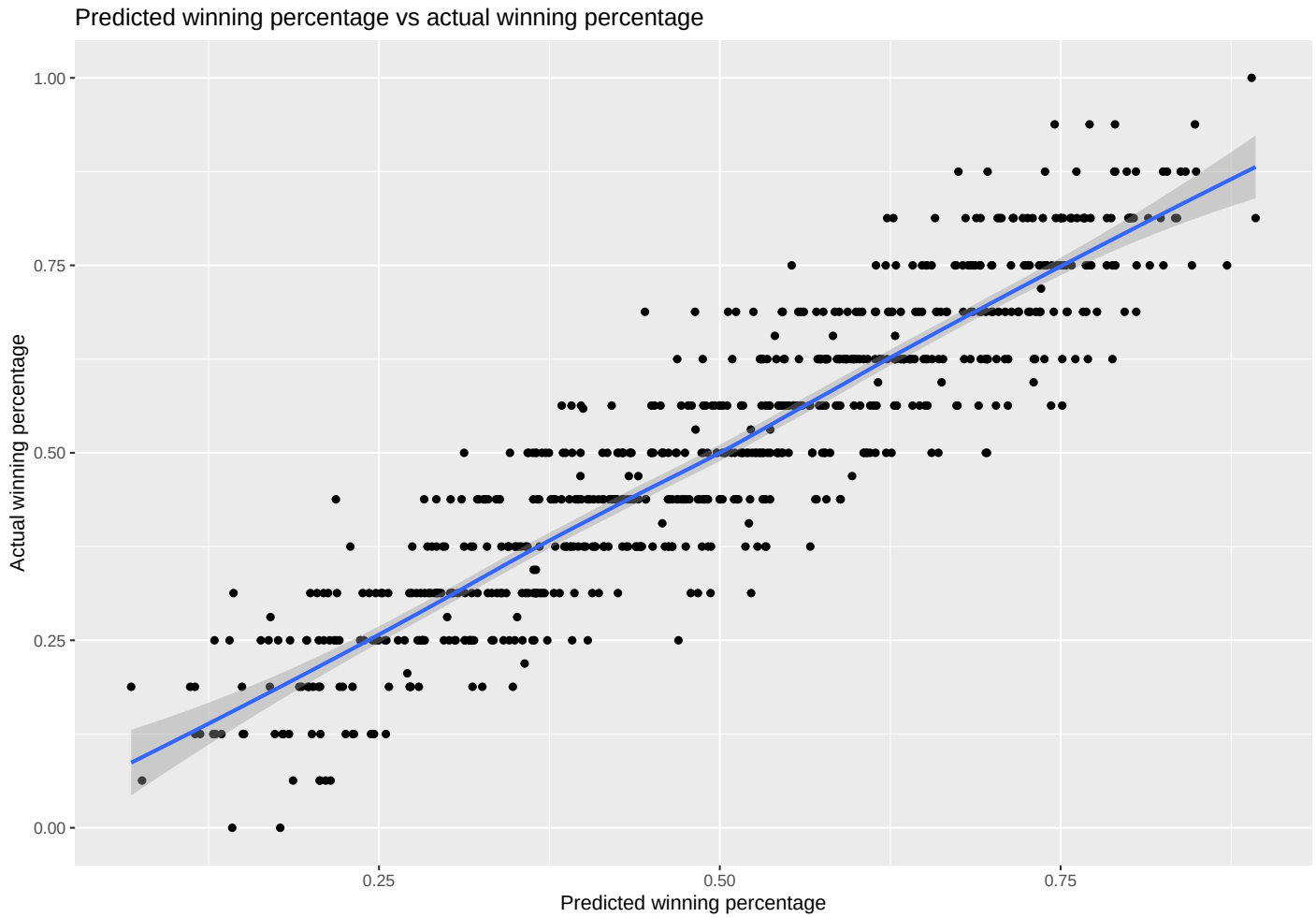
Getting a value for k.

```
k <- model$coefficients[1]
head(nfl_st <- nfl_st %>%
  mutate(PWPCT = ( PF ^ k / (PF ^ k + PA ^ k)), PW = (W + L + T) * PWPCT),5)
```

```
## # A tibble: 5 x 19
## # Groups:   Season [1]
##   Tm          W     L     T  Wpct    PF    PA    PD   MoV   SoS   SRS  OSRS  DSRS
##   <chr>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 New Y~    12     4     0 0.75    416   266   150   9.4   1.8  11.2   5.4   5.8
## 2 Miami~    10     6     0 0.625   321   265    56   3.5   2.4   5.9  -0.7   6.5
## 3 Buffa~    10     6     0 0.625   400   333    67   4.2   1     5.2   4.4   0.7
## 4 New E~     9     7     0 0.563   337   329     8   0.5   3.4   3.9   1.1   2.8
## 5 India~     3    13     0 0.188   310   444  -134  -8.4   3    -5.4  -0.1  -5.3
## # ... with 6 more variables: Season <dbl>, Conf <chr>, Div <chr>, Rank <int>,
## #   PWPCT <dbl>, PW <dbl>
```

Pythagorean Wins

```
nfl_st %>%
  ggplot(aes(x = PWPCT, Wpct)) +
  geom_point() +
  geom_smooth() +
  ggtitle("Predicted winning percentage vs actual winning percentage") +
  xlab("Predicted winning percentage") +
  ylab("Actual winning percentage")
```



Elo Ratings

Elo Ratings

Calculating elos for every season, with seasonal metrics

```
home_factor_nhl <- 50
k_nhl <- 6
starting_elos <- 1500

calculate_elos <- function(data, home_factor, k, starting_elos){
  relative_elos <- data.frame()
  final_elos <- data.frame()
  for (season in unique(nhl_games$season)){
  seasonal_data <- data %>%
    filter(season == season)
  season_elos <- elo.run(score(home_goals, away_goals) ~
          adjust(home_team_abr, home_factor_nhl) + away_team_abr,
          data = seasonal_data, k = k,
          initial.elos = starting_elos
        )
  starting_elos <- final.elos(season_elos) * 0.8
  relative_elos <- rbind(relative_elos,
                         data.frame(season_elos, season = season))
  final_elos_df <- data.frame(season = season,
                              team_abr = names(starting_elos),
                              elo = starting_elos)
  final_elos <- rbind(final_elos, final_elos_df)
  }
  return (list(final_elos=final_elos, relative_elos=relative_elos))
}

nhl_relative_elos <- calculate_elos(nhl_games, home_factor_nhl, k_nhl,
                                    starting_elos)$relative_elos

nhl_final <- calculate_elos(nhl_games, home_factor_nhl, k_nhl, starting_elos)$final_elos
nhl_relative_elos$Predicted <- ifelse(nhl_relative_elos$p.A > 0.5, 1,0)
```

Elo Ratings

Checking predictive power.

```
calculate_brier <- function(data){
  diff <- data$p.A - data$wins.A
  accuracy <- sum(diff^2) / nrow(data)
  return(accuracy)
}

(caret::confusionMatrix(data = factor(nhl_relative_elos$Predicted),
                        reference = factor(nhl_relative_elos$wins.A)))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction      0    0.5      1
##        0    59533   2852  48266
##        0.5      0      0      0
##        1   150835   8206 207379
##
## Overall Statistics
##
##                Accuracy : 0.5595
##                  95% CI : (0.5581, 0.5609)
##     No Information Rate : 0.5359
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.0939
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: 0 Class: 0.5 Class: 1
## Sensitivity            0.2830    0.00000   0.8112
## Specificity            0.8083    1.00000   0.2817
## Pos Pred Value         0.5380        NaN   0.5660
## Neg Pred Value         0.5884    0.97682   0.5638
```

Elo Ratings

Trying to play with k parameter to improve the accuracy.

```
nhl_k_picker <- function(data = nhl_games){
  result <- c()
  for( k in seq(4,12,1)){
    elos_for_k <- calculate_elos(data = data,
                                 home_factor_nhl,
                                 k,
                                 1500)

    br_scores <- elos_for_k$relative_elos %>%
      group_by(season) %>%
      summarise(br.score = sum((p.A - wins.A)^2) / n(),
                k = k)
    result <- rbind(result, br_scores)
  }
  model <- lm( br.score ~ 0 + k, data = result )
  return( list(model = model,
               df = as.data.frame(result)) )
}
```

Elo Ratings

```
k_list <- nhl_k_picker()

k_model <- k_list$model
k_df <- k_list$df
summary(k_model)
```

```
##
## Call:
## lm(formula = br.score ~ 0 + k, data = result)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.08510 -0.03144  0.02225  0.07615  0.13048
##
## Coefficients:
##    Estimate Std. Error t value Pr(>|t|)
## k 0.0269078  0.0006665   40.37   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.07326 on 170 degrees of freedom
## Multiple R-squared:  0.9056, Adjusted R-squared:  0.905
## F-statistic:  1630 on 1 and 170 DF,  p-value: < 2.2e-16
```

Elo Ratings

```
k_df %>%
  group_by(season) %>%
  filter(br.score == min(br.score)) %>%
  ungroup() %>%
  pull(k)
```

```
##  [1] 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
```

Elo Ratings

```
k_df <- k_df %>%
  group_by(k) %>%
  summarize(br.score = mean(br.score))

ggplot(k_df, aes(x = k, y = br.score) ) +
  geom_line() +
  ggtitle("Predictions accuracy using brier score vs k value") +
  xlab("K value") +
  ylab("Brier score")
```

